UNIVERSITÉ ALIOUNE DIOP DE BAMBEY

UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle

Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

Secur-SN

Intelligence Sécuritaire & Prédiction des Accidents de la Route

Sécurité Publique & Prévention Routière

Durée	4 semaines
Équipe	2 étudiants
Domaine	Sécurité Publique & Prévention Routière
Standard	M2 Approche Industrie | New Deal Technologique Sénégal
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • MLlib • Privacy by Design

1. Contexte et Objectifs

Le Sénégal enregistre ~600 morts/an sur les routes (GAMA/CETUD) et des taux de criminalité concentrés dans certains quartiers de Dakar et périphérie. Ce projet construit un système de prédiction des zones à risque d'accidents (hot-spots) et d'analyse des tendances criminelles, pour optimiser le déploiement des forces de sécurité et les campagnes de prévention.

> ℹ️ **New Deal Technologique Sénégal**
> Ce projet s'inscrit dans la Stratégie Sénégal Numérique 2025-2035 et le Plan Sénégal Émergent. L'objectif est de former des ingénieurs Big Data capables de construire des solutions Data-Driven directement applicables aux défis de développement du Sénégal.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingestion multi-sources**	rapports GAMA (accidents), données CETUD (transport), météo
2	**Prédiction des hot-spots d'accidents routiers (grille géospatiale 2km²)**	Prédiction des hot-spots d'accidents routiers (grille géospatiale 2km²)
3	**Analyse temporelle des incidents par zone, heure et type de véhicule**	Analyse temporelle des incidents par zone, heure et type de véhicule
4	**Privacy by Design**	anonymisation des victimes et des témoins (données sensibles)
5	**Dashboard CETUD**	carte des zones à risque, recommandations patrouilles préventives

1.2 Architecture End-to-End

📄 Architecture

```
[ SOURCES ]  ──▶  [ NiFi ]  ──▶  [ KAFKA ]  ──▶  [ SPARK ML ]  ──▶  [ STORAGE ]
 GAMA/CETUD     Orchestration    Topic:           Prédiction          HBase/HDFS
 Météo           & Routage        secur_incidents  Hot-spots           Gold Layer
 GPS/Cartow                       secur_meteo      Classification          │
                                                          │           [ ANALYTICS ]
                                                          ▼             Apache Hive
                                              [ ALERT LAYER ]        vue_hotspots
                                              Zone ROUGE             vue_tendances
```

2. Prérequis & Infrastructure

Configuration minimale

RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 requis par Spark.

setup.sh

```
# Démarrage infrastructure
mkdir -p data dags models nifi_templates
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Démarrer le simulateur
python kafka_producer_secur_sn.py &

# Interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

requirements.txt

```
# requirements.txt — Secur-SN
pyspark==3.3.2
kafka-python==2.0.2
pandera==0.17.2
pandas==2.0.3
apache-airflow==2.7.3
mlflow==2.8.1
pyhive==0.7.0
happybase==1.2.0
matplotlib==3.7.3
seaborn==0.12.2
faker==30.1.0
scipy==1.11.4
numpy==1.24.4
```

3. Codes Sources Complets & Fonctionnels

3.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Crée les 3 tables HBase de Secur-SN. À exécuter avant le démarrage du pipeline.

📄 **hbase_setup.py**

```
# hbase_setup.py — Tables HBase Secur-SN
import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_secur_sn_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        b'secur:incidents_temps_reel': {'b'meta': {'max_versions': 1}},  # Incidents en
cours (TTL 24h)
        b'secur:hotspots': {'b'meta': {'max_versions': 1}},  # Zones à risque avec score
(MAJ horaire)
        b'secur:stats_zone': {'b'meta': {'max_versions': 1}},  # Statistiques cumulées par
zone
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_secur_sn_tables()
```

3.2 Simulateur de Données — kafka_producer

> ℹ️ **Données simulées réalistes**
> Le simulateur génère des données calées sur les réalités de Sécurité Publique & Prévention Routière au Sénégal. Les champs PII (noms, IDs) sont présents dans le simulateur et seront anonymisés par la Privacy Layer Spark.

📄 **kafka_producer_secur_sn.py**

```
# kafka_producer_secur.py — Simulateur incidents routiers + criminalité
from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime
import numpy as np

random.seed(42); np.random.seed(42)
ZONES =
['DAKAR_PLATEAU','DAKAR_BANLIEUE','PIKINE','GUEDIAWAYE','RUFISQUE','THIES','KAOLACK']
TYPES_INCIDENT = ['ACCIDENT_MORTEL','ACCIDENT_GRAVE','ACCIDENT_LEGER',
                  'INFRACTION_CODE_ROUTE','VEHICULE_EPAVE']
TYPES_VEHICULE = ['MOTO_JAKARTA','CAR_RAPIDE','CAMION','VOITURE','BUS_DDD','CHARETTE']
# Coordonnées GPS approximatives par zone
GPS_ZONES = {
    'DAKAR_PLATEAU': (14.693,  -17.447),
```

```
    'DAKAR_BANLIEUE':(14.720,  -17.460),
    'PIKINE':        (14.751,  -17.390),
    'GUEDIAWAYE':    (14.770,  -17.385),
    'RUFISQUE':      (14.715,  -17.270),
    'THIES':         (14.791,  -16.935),
    'KAOLACK':       (14.150,  -16.072),
}
# Probabilité d'accident mortel par type de véhicule
RISK_VEHICULE = {'MOTO_JAKARTA':0.35,'CAR_RAPIDE':0.25,'CAMION':0.20,
                 'VOITURE':0.10,'BUS_DDD':0.06,'CHARETTE':0.04}

producer = KafkaProducer(bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode())

def gen_incident():
    zone     = random.choice(ZONES)
    gps_base = GPS_ZONES[zone]
    vehicule = random.choices(list(RISK_VEHICULE.keys()),
                              weights=list(RISK_VEHICULE.values()))[0]
    h = datetime.utcnow().hour
    # Accidents plus fréquents 7-9h et 17-21h (heures de pointe)
    facteur_heure = 1.5 if (7<=h<=9 or 17<=h<=21) else 0.8
    type_inc = random.choices(TYPES_INCIDENT,
                   weights=[0.05,0.15,0.40,0.35,0.05])[0]
    return {
        'incident_id':    str(uuid.uuid4()),
        'nom_victime':    f'VICTIME_{uuid.uuid4().hex[:6]}',  # Sera supprimé
        'tel_temoin':     f'+221 7{random.randint(10,99)} {random.randint(100,999)}
{random.randint(10,99):02d} {random.randint(10,99):02d}',  # Supprimé
        'zone':           zone,
        'type_incident':  type_inc,
        'type_vehicule':  vehicule,
        'latitude':       round(gps_base[0] + np.random.normal(0, 0.01), 6),
        'longitude':      round(gps_base[1] + np.random.normal(0, 0.01), 6),
        'nb_victimes':    random.choices([0,1,2,3,5],[50,25,15,7,3])[0],
        'heure':          h,
        'facteur_heure':  facteur_heure,
        'timestamp':      datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

if __name__ == '__main__':
    print('Simulateur Secur-SN démarré...')
    while True:
        producer.send('secur_incidents_raw', gen_incident())
        producer.flush(); time.sleep(4)
```

3.3 Pipeline Spark Streaming — streaming

> ⚠️ **Privacy critique**
> Drop des champs PII après anonymisation SHA-256. Les données sensibles ne doivent jamais apparaître dans HBase ou Hive.

streaming_secur_sn.py

```python
# streaming_secur.py — Pipeline Spark Streaming Secur-SN
# Score de risque géospatial + détection hot-spots
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, count, avg, sum as spark_sum,
    when, coalesce, current_timestamp
)
from pyspark.sql.types import *

SALT = os.environ.get('SECUR_SECRET_SALT','default_salt')
spark = SparkSession.builder.appName('Secur_SN_Streaming') \
    .config('spark.sql.shuffle.partitions','4').getOrCreate()
spark.sparkContext.setLogLevel('WARN')

inc_schema = StructType([
    StructField('incident_id',   StringType(), True),
    StructField('nom_victime',   StringType(), True),
    StructField('tel_temoin',    StringType(), True),
    StructField('zone',          StringType(), True),
    StructField('type_incident', StringType(), True),
    StructField('type_vehicule', StringType(), True),
    StructField('latitude',      FloatType(),  True),
    StructField('longitude',     FloatType(),  True),
    StructField('nb_victimes',   IntegerType(),True),
    StructField('heure',         IntegerType(),True),
    StructField('facteur_heure', FloatType(),  True),
    StructField('timestamp',     StringType(), True),
])

inc_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('subscribe','secur_incidents_raw').load()
    .select(from_json(col('value').cast('string'), inc_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
    # Privacy — suppression données victimes et témoins
    .withColumn('incident_secure', sha2(concat(col('incident_id'),lit(SALT)),256))
    .drop('incident_id','nom_victime','tel_temoin')
    # Score de gravité (MORTEL=5, GRAVE=3, LEGER=1)
    .withColumn('score_gravite',
        when(col('type_incident')=='ACCIDENT_MORTEL', lit(5.0))
        .when(col('type_incident')=='ACCIDENT_GRAVE',  lit(3.0))
        .when(col('type_incident')=='ACCIDENT_LEGER',  lit(1.0))
        .otherwise(lit(0.5)))
)

# Agrégation hot-spots (fenêtre 24h, watermark 2h)
hotspot_df = (inc_df
    .withWatermark('event_ts','2 hours')
    .groupBy(window('event_ts','24 hours','1 hour'), 'zone')
    .agg(
        count('incident_secure').alias('nb_incidents_24h'),
        spark_sum('score_gravite').alias('score_risque_total'),
        avg(coalesce('nb_victimes',lit(0))).alias('nb_victimes_moy'),
    )
    .withColumn('niveau_risque',
        when(col('score_risque_total') > 20, lit('ROUGE'))
        .when(col('score_risque_total') > 10, lit('ORANGE'))
        .otherwise(lit('VERT')))
)
```

```
q1 = (hotspot_df.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('topic','secur_alerts')
    .option('checkpointLocation','/tmp/secur_hotspot_ckpt')
    .outputMode('update').start())
q1.awaitTermination()
```

3.4 Tables Hive & Vues Analytics — hive_setup.sql

Les vues Hive utilisent systématiquement COALESCE sur les colonnes nullables pour éviter les NULL silencieux dans les agrégations.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Secur-SN
CREATE DATABASE IF NOT EXISTS secur_sn COMMENT 'Sécurité Sénégal — UADB 2025';
USE secur_sn;

CREATE TABLE IF NOT EXISTS incidents_historique (
    incident_secure STRING, zone STRING,
    type_incident STRING, type_vehicule STRING,
    score_gravite DOUBLE, nb_victimes INT,
    latitude DOUBLE, longitude DOUBLE, heure INT,
    ingestion_ts TIMESTAMP
) PARTITIONED BY (date_obs STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

-- Vue : Hot-spots par zone et heure
CREATE OR REPLACE VIEW vue_hotspots AS
SELECT zone, heure,
    COUNT(*)                                  AS nb_incidents,
    SUM(COALESCE(score_gravite,0))            AS score_risque,
    AVG(COALESCE(nb_victimes,0))              AS victimes_moy,
    CASE
        WHEN SUM(score_gravite) > 20 THEN 'ROUGE'
        WHEN SUM(score_gravite) > 10 THEN 'ORANGE'
        ELSE 'VERT'
    END AS niveau_risque
FROM incidents_historique
WHERE date_obs >= DATE_SUB(CURRENT_DATE(),30)
GROUP BY zone, heure
ORDER BY score_risque DESC;

-- Vue : Tendances par type de véhicule
CREATE OR REPLACE VIEW vue_tendances_vehicule AS
SELECT type_vehicule,
    COUNT(*)                  AS nb_incidents_total,
    AVG(score_gravite)        AS gravite_moy,
    SUM(nb_victimes)          AS total_victimes
FROM incidents_historique
GROUP BY type_vehicule
ORDER BY nb_incidents_total DESC;
```

3.5 DAG Airflow MLOps

Le BranchPythonOperator détecte automatiquement les dérives et déclenche le réentraînement du modèle si nécessaire. Les recommandations sont écrites dans HBase pour un accès temps réel.

dags/secur_sn_dag.py

```
# dags/secur_sn_dag.py — DAG Airflow Secur-SN
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('secur_dag')
default_args = {'owner':'seye_ahmed','retries':2,'retry_delay':timedelta(minutes=5)}
```

```
def check_hotspots(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore',port=10000)
    cur  = conn.cursor()
    cur.execute("SELECT COUNT(*) FROM secur_sn.vue_hotspots WHERE niveau_risque='ROUGE'")
    n = cur.fetchone()[0]
    return 'retrain_model' if n > 3 else 'update_hotspots'

def retrain_model(**ctx):
    subprocess.run(['spark-submit','--master','spark://spark-master:7077',
        '/opt/models/train_hotspot_model.py'],check=True,timeout=3600)

def update_hotspots(**ctx):
    import happybase; from pyhive import hive; from datetime import datetime
    conn_hbase = happybase.Connection('hbase',port=9090); conn_hbase.open()
    conn_hive  = hive.Connection(host='hive-metastore',port=10000)
    cur = conn_hive.cursor()
    cur.execute('SELECT zone, heure, score_risque, niveau_risque FROM
secur_sn.vue_hotspots LIMIT 100')
    table = conn_hbase.table(b'secur:hotspots')
    for row in cur.fetchall():
        zone,heure,score,niveau = row
        table.put(f'{zone}_H{heure}'.encode(),{
            b'stats:zone':zone.encode(), b'stats:score':str(score).encode(),
            b'stats:niveau':niveau.encode(),
b'stats:ts':datetime.utcnow().isoformat().encode()})
    conn_hbase.close()

with DAG('secur_sn_monitoring', default_args=default_args,
         schedule_interval='0 */2 * * *', start_date=days_ago(1), catchup=False,
         tags=['secur-sn','securite','hotspots']) as dag:
    start  = DummyOperator(task_id='start'); end = DummyOperator(task_id='end')
    branch = BranchPythonOperator(task_id='check_hotspots',
                                  python_callable=check_hotspots,provide_context=True)
    t_train=
PythonOperator(task_id='retrain_model',python_callable=retrain_model,provide_context=True)
    t_upd  =
PythonOperator(task_id='update_hotspots',python_callable=update_hotspots,provide_context=T
rue)
    start >> branch >> [t_train, t_upd]
    t_train >> t_upd >> end; t_upd >> end
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet (GitHub) + template NiFi. Données visibles dans HDFS, topic Kafka secur_incidents_raw actif.
2	**Modèle ML & MLOps**	Script d'entraînement + RMSE/R^2 documentés + DAG Airflow avec BranchPythonOperator fonctionnel.
3	**Catalogue Hive**	Tables créées, vues fonctionnelles avec au moins 1 alerte visible dans HBase en démonstration.
4	**Dashboard & Recommandations**	Dashboard graphique (Matplotlib) + recommandations concrètes contextualisées au Sécurité Publique & Prévention Routière sénégalais.

5. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Chaque critère est évalué selon 4 niveaux de maîtrise. La démonstration live en soutenance est obligatoire. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Sources	**3 pts**	NiFi absent	1 source/3	2 sources + topics	GAMA+CETUD+météo, 2 topics distincts, messages géolocalisés
Validation & Privacy (Victimes)	**4 pts**	Données victimes en clair	Schéma sans anonymisation victime	SHA-256 sans drop noms victimes	Pandera + drop(nom_victime, tel_temoin) + 0 PII HBase — conformité données sensibles
Streaming & Détection Hot-Spots	**6 pts**	Pas de Spark	Lecture seule	Fenêtres 24h + comptage basique	Score de risque géospatial (densité×gravité×météo) + watermarks
Prédiction & Analytics	**4 pts**	Pas de modèle	Règles fixes	RF entraîné sans DAG	Random Forest hot-spots + DAG quotidien + matrice confusion documentée
Dashboard & Recommandations CETUD	**3 pts**	Aucun dashboard	KPIs sans géolocalisation	Carte zones + pas de recommandation	Carte hot-spots + recommandations patrouilles + alertes météo-risque
TOTAL	**20 pts**				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + captures d'écran	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
Tests pytest sur les schémas Pandera et les UDFs	+0.5 pt
Démo live d'une alerte HBase sans erreur en soutenance	+0.5 pt

🛡️ **Bon courage pour Secur-SN !**

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel